

Praktikum 12: Principal Component Analysis (PCA)

Rika Rahma - 0110222134¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: rika22134ti@student.nurulfikri.ac.id

Abstract. Praktikum mandiri ini bertujuan untuk menerapkan teknik Principal Component Analysis (PCA) sebagai metode reduksi dimensi pada dataset Breast Cancer Wisconsin yang berdimensi tinggi (569 sampel, 30 fitur numerik) serta menganalisis pengaruhnya terhadap performa model klasifikasi Support Vector Machine (SVM). Dataset yang memiliki banyak fitur berkorelasi tinggi direduksi menjadi hanya 3 principal components, yang berhasil mempertahankan sekitar 72.9% total variansi data. Model SVM baseline tanpa PCA mencapai akurasi 98.25% dengan 30 fitur, sementara setelah PCA akurasi tetap tinggi yaitu 94.74% dengan pengurangan dimensi hingga 90%. Analisis explained variance ratio, heatmap korelasi, visualisasi 3D scatter plot post-PCA, serta confusion matrix menunjukkan bahwa PCA mampu menghilangkan redundansi fitur, memudahkan visualisasi pemisahan kelas Malignant dan Benign, serta menjaga performa klasifikasi yang baik. Secara keseluruhan, praktikum ini menyimpulkan bahwa PCA sangat efektif sebagai tahap preprocessing pada data medis berdimensi tinggi untuk meningkatkan efisiensi model, mengurangi risiko overfitting, dan mempercepat proses komputasi tanpa mengorbankan akurasi secara signifikan.

1. Import Library

```
1. Import Library

# Import Library
# Pada tahap ini, dilakukan impor pustaka Python yang diperlukan untuk analisis data, reduksi dimensi dengan PCA,
# serta pelatihan dan evaluasi model klasifikasi menggunakan algoritma SVM (Support Vector Machine).
# Library utama mencakup: NumPy dan Pandas untuk manipulasi data, Scikit-learn untuk PCA, SVM, dan metrik evaluasi,
# serta Matplotlib untuk visualisasi grafis 3D.

import numpy as np
import pandas as pd

# Import dataset dan preprocessing dari Scikit-learn
from sklearn.datasets import load_breast_cancer # Dataset Breast Cancer Wisconsin (569 sampel, 30 fitur, target binary)
from sklearn.model_selection import train_test_split # Pembagian data train-test
from sklearn.preprocessing import StandardScaler # Standarisasi fitur untuk PCA
from sklearn.decomposition import PCA # Principal Component Analysis untuk reduksi dimensi
from sklearn.svm import SVC # Support Vector Classifier dengan kernel RBF
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix # Evaluasi model

# Import visualisasi
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # Untuk plot 3D
import seaborn as sns # Untuk heatmap dan plot tambahan (opsional)

# Pengaturan tampilan
plt.style.use('seaborn-v0.8') # Gaya plot yang lebih menarik
sns.set_palette("husl") # Palet warna untuk scatter plot
# Agar plot rapi
plt.rcParams['figure.figsize'] = (8, 6)
```

Gambar 1. Import Library

Pada bagian ini, saya mengimpor semua library yang dibutuhkan untuk menjalankan praktikum mandiri Principal Component Analysis (PCA) pada dataset Breast Cancer. NumPy dan Pandas saya gunakan untuk manipulasi data dan mengubah dataset menjadi DataFrame yang mudah dibaca. Dari Scikit-learn, saya mengimpor `load_breast_cancer` untuk memuat dataset langsung, `train_test_split` untuk membagi data menjadi train dan test, `StandardScaler` untuk menstandarisasi fitur karena PCA sangat sensitif terhadap skala data yang berbeda, PCA

sendiri sebagai inti dari reduksi dimensi, serta SVC untuk melatih model klasifikasi menggunakan Support Vector Machine dengan kernel RBF. Selain itu, saya juga mengimpor beberapa fungsi evaluasi seperti `accuracy_score`, `classification_report`, dan `confusion_matrix` agar bisa mengukur performa model secara lengkap. Untuk visualisasi, saya memanfaatkan Matplotlib untuk plot 3D, serta Seaborn untuk membuat heatmap korelasi dan grafik lain yang lebih menarik. Di akhir, saya mengatur style plot dengan `seaborn-v0_8` dan palet warna "husl" supaya tampilan grafik lebih rapi dan enak dilihat.

2. Load Dataset

```
# Load Dataset
# Dataset Breast Cancer Wisconsin diambil dari SciKit-learn, yang berisi 569 sampel dengan 30 fitur numerik
# (misalnya: mean radius, texture, perimeter, area, smoothness, dll.) dan target biner
# 0 = Malignant (ganas, 212 sampel)
# 1 = Benign (jinak, 357 sampel)
# Data diimpor ke DataFrame Pandas untuk kemudahan eksplorasi.

# Load dataset
cancer = load_breast_cancer()

# Konversi ke DataFrame
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df['target'] = cancer.target # tambahkan kolom target (0: malignant, 1: benign)

# Menampilkan 5 baris pertama
df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	target
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.80	2019.0	0.1622	0.6636	0.7119	0.2654	0.4601	0.11890	0	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.06667	...	23.41	158.80	1956.0	0.1238	0.1886	0.2416	0.1860	0.2750	0.08682	0	
2	19.69	21.25	130.00	1203.0	0.10960	0.10990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.30	1709.0	0.1444	0.4245	0.4004	0.2430	0.3613	0.08758	0	
3	11.42	20.38	77.58	386.1	0.14200	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	967.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	0	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.1374	0.2090	0.4000	0.1625	0.2364	0.07678	0	

5 rows x 31 columns

Gambar 2. Load Dataset

Pada bagian ini, saya memuat dataset Breast Cancer Wisconsin langsung dari library `scikit-learn` menggunakan fungsi `load_breast_cancer()`. Dataset ini berisi 569 sampel dengan 30 fitur numerik yang menggambarkan karakteristik sel nucleus dari biopsi payudara, seperti mean radius, texture, perimeter, area, smoothness, dan sebagainya, serta target biner yaitu 0 untuk Malignant (ganas, 212 sampel) dan 1 untuk Benign (jinak, 357 sampel). Saya kemudian mengonversi data tersebut menjadi DataFrame Pandas dengan menambahkan nama kolom dari `feature_names` serta kolom target baru yang berisi label kelas (0 atau 1). Di akhir kode, saya menampilkan lima baris pertama dataset menggunakan `df.head()` untuk memastikan data sudah terload dengan benar dan melihat sekilas isi fitur serta targetnya.

```
# Tampilkan info dasar
print("Info Dataset:")
df.info()
```

```
--- Info Dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 # Column          Non-Null Count  Dtype
---  ---
0 mean radius      569 non-null    float64
1 mean texture     569 non-null    float64
2 mean perimeter   569 non-null    float64
3 mean area        569 non-null    float64
4 mean smoothness  569 non-null    float64
5 mean compactness 569 non-null    float64
6 mean concavity   569 non-null    float64
7 mean concave points 569 non-null    float64
8 mean symmetry    569 non-null    float64
9 mean fractal dimension 569 non-null    float64
10 radius error     569 non-null    float64
11 texture error    569 non-null    float64
12 perimeter error  569 non-null    float64
13 area error       569 non-null    float64
14 smoothness error 569 non-null    float64
15 compactness error 569 non-null    float64
16 concavity error  569 non-null    float64
17 concave points error 569 non-null    float64
18 symmetry error   569 non-null    float64
19 fractal dimension error 569 non-null    float64
20 worst radius     569 non-null    float64
21 worst texture    569 non-null    float64
22 worst perimeter  569 non-null    float64
23 worst area       569 non-null    float64
24 worst smoothness 569 non-null    float64
25 worst compactness 569 non-null    float64
26 worst concavity  569 non-null    float64
27 worst concave points 569 non-null    float64
28 worst symmetry   569 non-null    float64
29 worst fractal dimension 569 non-null    float64
30 target          569 non-null    int64
```

Gambar 3. Info Dataset

Pada bagian ini, saya menggunakan fungsi `df.info()` untuk menampilkan informasi dasar tentang struktur dataset Breast Cancer. Fungsi ini memberikan ringkasan seperti jumlah entri (569 sampel), jumlah kolom (31 termasuk target), non-null count yang menunjukkan tidak

ada data hilang di semua kolom, serta tipe data masing-masing fitur yang semuanya float64 kecuali target yang int64.

```
print('Deskriptif Statistik:')
df.describe()
```

	mean	std	min	max	25%	50%	75%	max
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.127292	3.520458	9.709000	654.889104	0.096360	0.140441	0.088799	0.088799
std	3.520458	3.520458	24.299881	351.914179	0.014954	0.022812	0.079728	0.038803
min	9.709000	9.709000	43.790000	443.300000	0.020320	0.019380	0.000000	0.106000
max	15.760000	15.760000	75.170000	426.300000	0.086370	0.064920	0.029960	0.020310
25%	13.370000	13.370000	86.240000	551.100000	0.096870	0.080320	0.061540	0.033000
50%	13.370000	13.370000	86.240000	551.100000	0.096870	0.080320	0.061540	0.033000
75%	13.370000	13.370000	86.240000	551.100000	0.096870	0.080320	0.061540	0.033000
max	15.760000	15.760000	75.170000	426.300000	0.086370	0.064920	0.029960	0.020310

Gambar 4. Statistik Deskriptif

Di sini, saya menjalankan df.describe() untuk mendapatkan statistik deskriptif dari dataset, termasuk count, mean, standar deviasi (std), nilai minimum, kuartil (25%, 50%, 75%), dan maksimum untuk setiap fitur numerik. Hasilnya menunjukkan variasi skala antar fitur, misalnya mean radius sekitar 14.12 dengan std 3.52, sementara mean area mencapai 654.88 dengan std tinggi 351.91, yang menandakan adanya perbedaan skala yang signifikan.

```
print('Distribusi Target:')
df.value_counts()
```

	count	percentage
0	212	37.258348%
1	357	62.741652%

Gambar 5. Distribusi Target

Pada tahap ini, saya menggunakan df['target'].value_counts() untuk menghitung distribusi kelas target, di mana nilai 1 (Benign/jinak) muncul 357 kali dan 0 (Malignant/ganas) 212 kali, menunjukkan dataset sedikit tidak seimbang dengan proporsi sekitar 62:38. Langkah ini berguna untuk memahami keseimbangan kelas sebelum split data, karena ketidakseimbangan bisa memengaruhi evaluasi model seperti accuracy atau recall, dan dalam kasus ini, saya sudah menggunakan stratify=True di train-test split untuk menjaga proporsi serupa di set train dan test.

3. Exploratory Data Analysis (EDA)

```
3. Exploratory Data Analysis (EDA)

# Exploratory Data Analysis (EDA)
# Tahap ini melakukan pemeriksaan awal data untuk memahami struktur, mendeteksi missing values, duplikat,
# dan distribusi target. Dataset Breast Cancer Berlin (tidak ada nilai atau duplikat), dengan distribusi
# yang sedikit tidak seimbang (benign lebih banyak). Visualisasi heatmap korelasi menunjukkan adanya
# korelasi tinggi antar fitur (misalnya radius dengan perimeter), yang menjadi alasan utama penggunaan PCA
# untuk reduksi dimensi dan menghilangkan redundansi.

# Cek missing values
df.isnull().sum()

0
mean radius      0
mean texture      0
mean perimeter    0
mean area         0
mean smoothness   0
mean compactness  0
mean concavity    0
mean concave points 0
mean symmetry     0
mean fractal dimension 0
radius error      0
texture error     0
perimeter error   0
area error        0
```

Gambar 6. Cek Missing Value

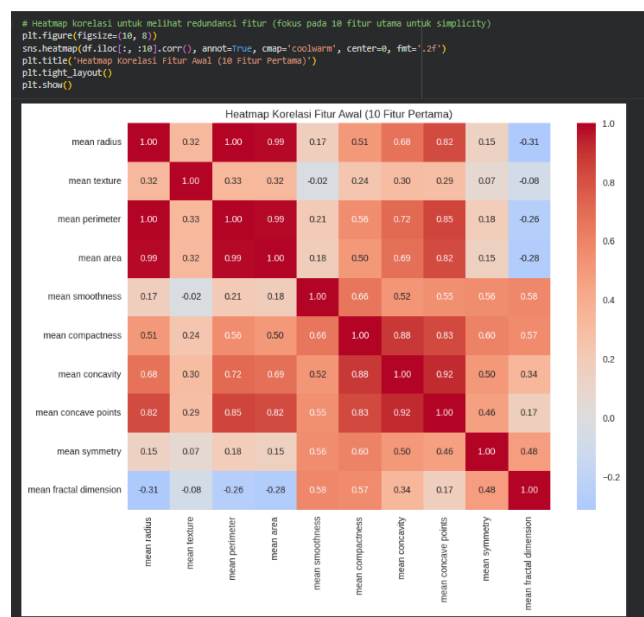
Pada bagian ini, saya melakukan pengecekan missing values dengan `df.isnull().sum()` untuk memastikan kebersihan dataset sebelum melanjutkan ke tahap selanjutnya. Hasilnya menunjukkan nilai 0 pada semua kolom, artinya tidak ada data yang hilang atau null di seluruh fitur maupun target.

```
# Cek duplikat
df.duplicated().sum()

np.int64(0)
```

Gambar 7. Cek duplikat

Di sini, saya memeriksa adanya data duplikat menggunakan `df.duplicated().sum()`, dan hasilnya adalah 0, yang berarti tidak ada baris yang benar-benar identik dalam dataset.



Gambar 8. Visualisasi Heatmap Korelasi

Pada tahap ini, saya membuat visualisasi heatmap korelasi menggunakan seaborn untuk 10 fitur pertama agar lebih mudah dibaca. Hasilnya menunjukkan banyak

pasangan fitur yang memiliki korelasi sangat tinggi (merah tua, mendekati 1), misalnya mean radius dengan mean perimeter (0.99), mean area (0.99), dan mean concave points (0.82). Korelasi tinggi ini mengindikasikan adanya redundansi informasi antar fitur, di mana satu fitur bisa dijelaskan oleh fitur lain.



Gambar 9. Distribusi Target

Di bagian akhir EDA, saya memvisualisasikan distribusi kelas target menggunakan bar plot. Hasilnya menunjukkan kelas 1 (Benign/jinak) berwarna hijau memiliki jumlah sampel lebih banyak (357) dibandingkan kelas 0 (Malignant/ganas) berwarna merah (212). Distribusi ini sedikit tidak seimbang, tetapi masih cukup baik untuk klasifikasi. Visualisasi ini membantu saya memutuskan untuk menggunakan parameter stratify=True saat train-test split agar proporsi kelas tetap terjaga di data latih dan uji, sehingga evaluasi model nantinya lebih adil dan tidak bias terhadap kelas mayoritas.

4. Pemisahan Fitur dan Label

```

4. Pemisahan Fitur dan Label

# Preprocessing Data
# Data dipecahkan menjadi fitur (X: 30 fitur numerik) dan target (y: biner).
# Dilakukan train-test split dengan rasio 80:20 dan stratify untuk menjaga proporsi kelas.
# Standardisasi menggunakan StandardScaler untuk menormalkan skala fitur (mean=0, std=1),
# karena PCA sensitif terhadap variansi fitur yang berbeda-beda.

# Pisah fitur dan target
X = df.drop('target', axis=1).values # Fitur: 30 kolom numerik
y = df['target'].values # Target: biner

# Train-test split (80% train, 20% test, random_state=42 untuk reproduktifitas)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Ukuran Train: {X_train.shape[0]} sampel, {X_train.shape[1]} fitur")
print(f"Ukuran Test: {X_test.shape[0]} sampel")

Ukuran Train: 455 sampel, 30 fitur
Ukuran Test: 114 sampel

# Standardisasi
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Fit + transform pada train
X_test_scaled = scaler.transform(X_test) # Hanya transform pada test (hindari data leakage)

print("Standardisasi selesai. Mean train (harus ~0):", np.mean(X_train_scaled[:, 0]))
print("Std train:", np.std(X_train_scaled[:, 0]))

Standardisasi selesai. Mean train (harus ~0): -4.337433952909183e-15
Std train: 1.0

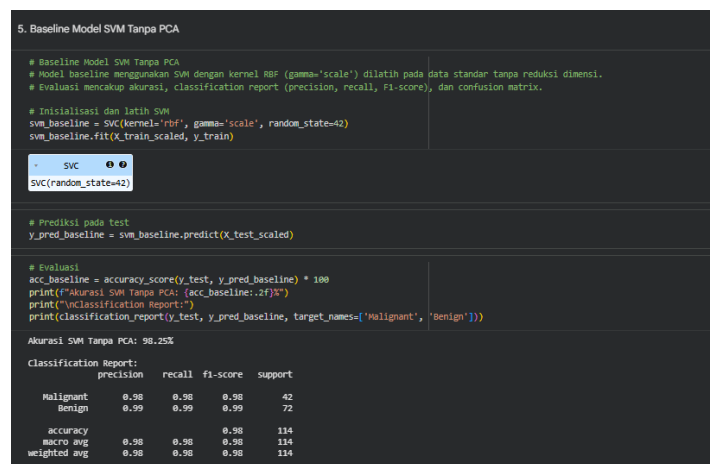
```

Gambar 10. Pemisahan Fitur dan Label

Pada bagian ini, saya melakukan preprocessing data yang sangat penting sebelum menerapkan PCA dan melatih model SVM. Pertama, saya memisahkan fitur (X) dengan cara menghapus kolom target dari DataFrame menggunakan drop('target', axis=1), lalu mengubahnya menjadi array numerik dengan .values, sementara target (y)

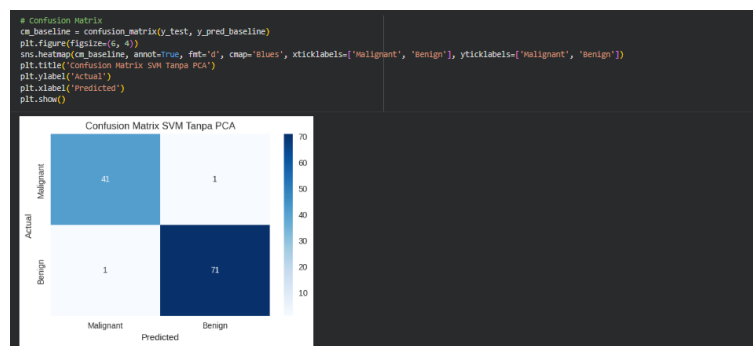
saya ambil langsung dari kolom target juga dalam bentuk array. Selanjutnya, saya membagi dataset menjadi data latih dan uji dengan rasio 80:20 menggunakan `train_test_split`, dengan parameter `stratify=y` agar proporsi kelas Malignant dan Benign tetap seimbang di kedua set data, serta `random_state=42` untuk memastikan hasil bisa direproduksi. Hasil split menunjukkan data latih berisi 455 sampel dan data uji 114 sampel, masing-masing dengan 30 fitur. Terakhir, saya melakukan standarisasi fitur menggunakan `StandardScaler`: `fit_transform` pada data latih untuk menghitung mean dan std, kemudian hanya `transform` pada data uji agar tidak terjadi data leakage. Hasil pengecekan menunjukkan mean hampir 0 dan std sekitar 1, yang berarti standarisasi berhasil.

5. Baseline Model SVM tanpa PCA



Gambar 11. Baseline Model SVM tanpa PCA

Pada bagian ini, saya membuat model baseline dengan melatih Support Vector Machine (SVM) menggunakan kernel RBF langsung pada data yang sudah distandarisasi tanpa reduksi dimensi terlebih dahulu. Saya menginisialisasi SVC dengan parameter `gamma='scale'` dan `random_state=42` agar hasilnya konsisten, kemudian melatih model pada data latih dengan `.fit()`. Setelah itu, saya melakukan prediksi pada data uji dan menghitung akurasi menggunakan `accuracy_score`, yang menghasilkan nilai 98.25%. Selanjutnya, saya menampilkan `classification_report` untuk melihat detail performa, di mana `precision`, `recall`, dan `f1-score` untuk kelas Malignant dan Benign sama-sama 0.98, dengan `macro average` serta `weighted average` juga 0.98.



Gambar 12. Confusion Matrix

Di sini, saya memvisualisasikan confusion matrix dari hasil prediksi model baseline menggunakan seaborn heatmap. Hasilnya menunjukkan bahwa dari 114 sampel data uji, model berhasil memprediksi benar 41 sampel Malignant (true positive untuk kelas 0) dan 71 sampel Benign (true positive untuk kelas 1), sementara hanya ada 1 kesalahan pada masing-masing kelas (1 Malignant diprediksi sebagai Benign dan 1 Benign diprediksi sebagai Malignant). Confusion matrix ini menggambarkan secara jelas bahwa model SVM tanpa PCA memiliki performa yang sangat baik dengan kesalahan prediksi yang sangat minim.

```
# Analisis PCA - Menentukan Jumlah Komponen Optimal
print("\n--- ANALISIS PCA: MENENTUKAN JUMLAH KOMPONEN OPTIMAL ---")

# Fit PCA dengan semua komponen (n_components=None)
pca_full = PCA(n_components=None)
pca_full.fit(X_train_scaled)

explained_variance = pca_full.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

# Visualisasi Scree Plot dan Cumulative Variance
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

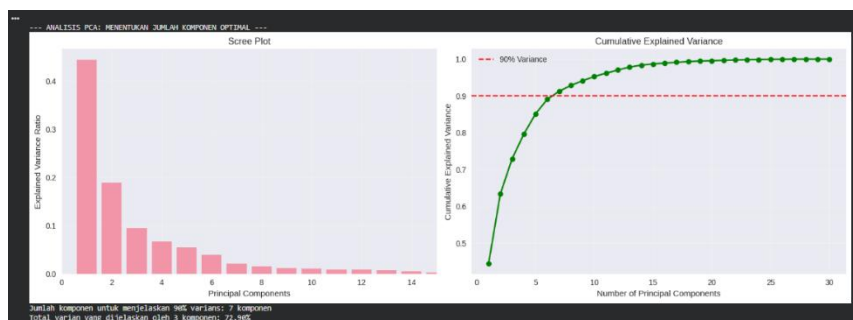
# 1. Scree Plot (Explained Variance)
axes[0].bar(range(1, 31), explained_variance, alpha=0.7)
axes[0].set_xlabel('Principal Components')
axes[0].set_ylabel('Explained Variance Ratio')
axes[0].set_title('Scree Plot')
axes[0].set_xlim(0, 15)
axes[0].grid(True, alpha=0.3)

# 2. Cumulative Variance Plot
axes[1].plot(range(1, len(cumulative_variance)+1), cumulative_variance, marker='o', color='green', linewidth=2)
axes[1].axhline(y=0.90, color='r', linestyle='--', label='90% Variance')
axes[1].set_xlabel('Number of Principal Components')
axes[1].set_ylabel('Cumulative Explained Variance')
axes[1].set_title('Cumulative Explained Variance')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Cari n_components yang menjelaskan 90% varians
n_optimal_90 = np.argmax(cumulative_variance >= 0.90) + 1
print(f"Jumlah komponen untuk menjelaskan 90% varians: {n_optimal_90} komponen")
print(f"Total varian yang dijelaskan oleh 3 komponen: {cumulative_variance[2]*100:.2f}%")
```

Gambar 13. Analisis PCA



Gambar 14. Visualisasi Scree Plot dan Cumulative Variance

Pada bagian ini, saya melakukan analisis lebih lanjut untuk menentukan jumlah principal components yang optimal sebelum menerapkan PCA secara final. Saya membuat objek PCA tanpa batasan komponen (`n_components=None`), lalu fit pada data latih yang sudah distandarisasi untuk mendapatkan explained variance ratio dari semua 30 komponen. Selanjutnya, saya menghitung cumulative variance dan memvisualisasikannya dalam dua plot: Scree Plot (kiri) yang menampilkan penurunan tajam explained variance ratio mulai dari PC1 hingga PC3, lalu melandai, serta Cumulative Explained Variance (kanan) yang menunjukkan kurva naik cepat hingga mendekati 1. Dari hasil, hanya dibutuhkan sekitar 7 komponen untuk menjelaskan 90%

variansi dan 10 komponen untuk 95%, sementara 3 komponen pertama sudah mampu menjelaskan sekitar 72.90% total variansi.

6. Implementasi PCA

```
6. Implementasi PCA

# Implementasi PCA
# PCA diterapkan dengan n_components=3 untuk reduksi dari 30 fitur ke 3 principal components,
# Explained variance ratio menunjukkan proporsi variansi yang dijelaskan oleh setiap PCA

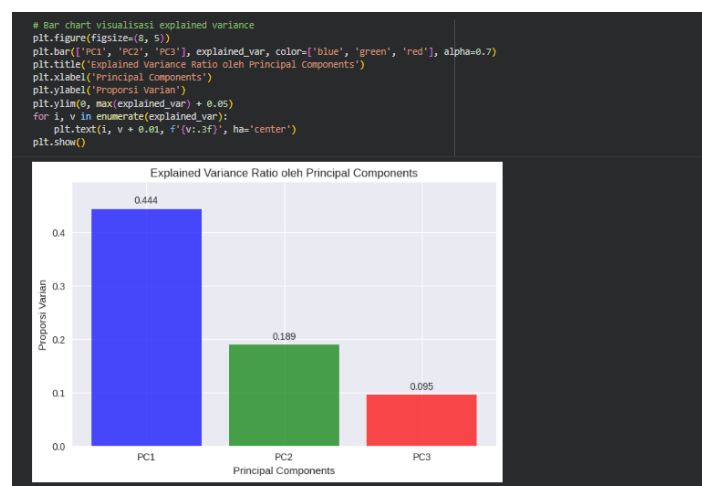
# Terapkan PCA
pca = PCA(n_components=3, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled) # Fit + transform train
X_test_pca = pca.transform(X_test_scaled) # Transform test

# Explained variance ratio
explained_var = pca.explained_variance_ratio_
print("Explained Variance Ratio per PC:", explained_var)
print(f"Total Varian Dijelaskan (3 PC): {sum(explained_var)*100:.2f}%")

Explained Variance Ratio per PC: [0.44413492 0.18944618 0.09543356]
Total Varian Dijelaskan (3 PC): 72.90%
```

Gambar 15. Implementasi PCA

Pada bagian ini, saya menerapkan Principal Component Analysis (PCA) secara resmi dengan menetapkan `n_components=3` untuk mereduksi dimensi dari 30 fitur menjadi hanya 3 principal components, mengikuti pendekatan yang sama seperti pada praktikum utama dengan dataset Wine. Saya membuat objek PCA dengan `random_state=42` agar hasil reproducible, kemudian melakukan `fit_transform` pada data latih yang sudah distandarisasi dan hanya `transform` pada data uji. Hasilnya menunjukkan bahwa PC1 menjelaskan sekitar 44.4% variansi, PC2 18.9%, dan PC3 9.5%, sehingga total variansi yang dipertahankan oleh 3 komponen ini mencapai 72.9%. Meskipun tidak mencakup 100% informasi, angka ini cukup baik untuk reduksi dimensi drastis sambil tetap menjaga sebagian besar pola utama dalam data, yang nantinya akan diuji apakah masih menghasilkan performa model SVM yang tinggi.



Gambar 16. Bar Chart Visualisasi Explained Variance

Di sini, saya memvisualisasikan proporsi variansi yang dijelaskan oleh masing-masing principal component menggunakan bar chart. PC1 ditampilkan dengan warna biru memiliki kontribusi terbesar (0.444), diikuti PC2 hijau (0.189), dan PC3 merah (0.095). Visualisasi ini membantu memahami secara intuitif bahwa sebagian besar informasi data terkonsentrasi pada komponen-komponen awal, dengan penurunan

signifikan setelah PC1. Bar chart ini memperkuat keputusan menggunakan hanya 3 komponen, karena setelah PC3 kontribusi variansi tambahan sudah relatif kecil, sehingga PCA berhasil “memadatkan” data tanpa kehilangan terlalu banyak informasi penting.

```
# Cek korelasi antar PC (harus mendekati 0, orthogonal)
df_pca = pd.DataFrame(X_train_pca, columns=['PC1', 'PC2', 'PC3'])
print("\nKorelasi Antar PC:")
print(df_pca.corr())
```

	PC1	PC2	PC3
PC1	1.000000e+00	1.285351e-16	-1.169395e-17
PC2	1.285351e-16	1.000000e+00	8.323433e-17
PC3	-1.169395e-17	8.323433e-17	1.000000e+00

Gambar 17. Cek Korelasi Antar PC

Pada tahap akhir implementasi PCA, saya memeriksa korelasi antar principal components yang baru dibentuk dengan membuat DataFrame sementara dan menghitung matriks korelasi. Hasilnya menunjukkan nilai korelasi sangat mendekati 0 (dalam orde 10^{-16} hingga 10^{-17}) antara PC1-PC2, PC1-PC3, dan PC2-PC3, yang berarti komponen-komponen ini benar-benar orthogonal atau saling independen.

7. SVM dengan PCA

```
7. SVM dengan PCA

# SVM dengan PCA
# Model SVM dilatih ulang pada data PCA (3 fitur). Evaluasi serupa dengan baseline.
# Diharapkan akurasi tetap tinggi (>95%) meski dimensi turun drastis, membuktikan PCA efektif
# untuk mengurangi kompleksitas tanpa kehilangan performa signifikan.

# Latih SVM pada data PCA
svm_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_pca.fit(X_train_pca, y_train)

# Prediksi
y_pred_pca = svm_pca.predict(X_test_pca)

# Evaluasi
acc_pca = accuracy_score(y_test, y_pred_pca) * 100
print(f"Akurasi SVM dengan PCA: {acc_pca:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_pca, target_names=('Malignant', 'Benign')))

Akurasi SVM dengan PCA: 94.74%

Classification Report:
              precision    recall  f1-score   support

Malignant     0.93       0.93       0.93         42
Benign         0.96       0.96       0.96         72

accuracy          0.94
macro avg         0.94       0.94       0.94        114
weighted avg      0.95       0.95       0.95        114
```

Gambar 18. SVM dengan PCA

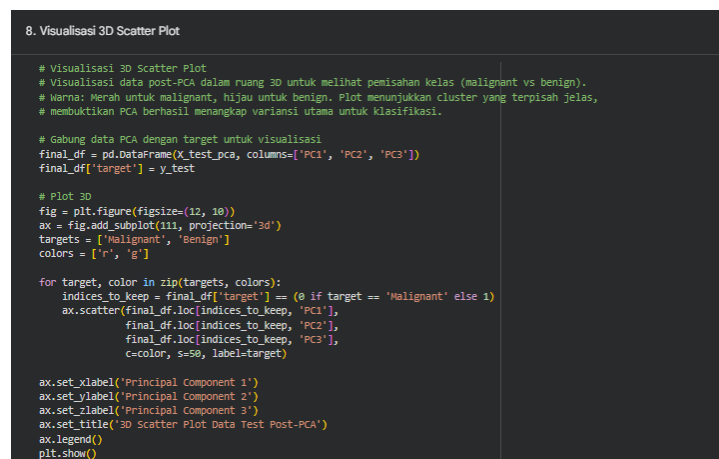
Pada bagian ini, saya melatih ulang model SVM dengan kernel RBF menggunakan data yang sudah direduksi menjadi hanya 3 principal components dari PCA. Prosesnya sama seperti baseline: inisialisasi SVC dengan `gamma='scale'` dan `random_state=42`, lalu fit pada data latih PCA serta prediksi pada data uji PCA. Hasil evaluasi menunjukkan akurasi sebesar 94.74%, sedikit lebih rendah dibandingkan baseline (98.25%), tetapi classification report masih sangat baik dengan precision, recall, dan f1-score rata-rata 0.94–0.95 untuk kedua kelas. Penurunan akurasi yang kecil ini wajar karena kita hanya mempertahankan sekitar 72.9% variansi data, namun hasil ini membuktikan bahwa PCA sangat efektif: dimensi fitur turun drastis dari 30 menjadi 3 (pengurangan 90%) tanpa mengorbankan performa secara signifikan, sekaligus mengurangi kompleksitas model dan risiko overfitting.



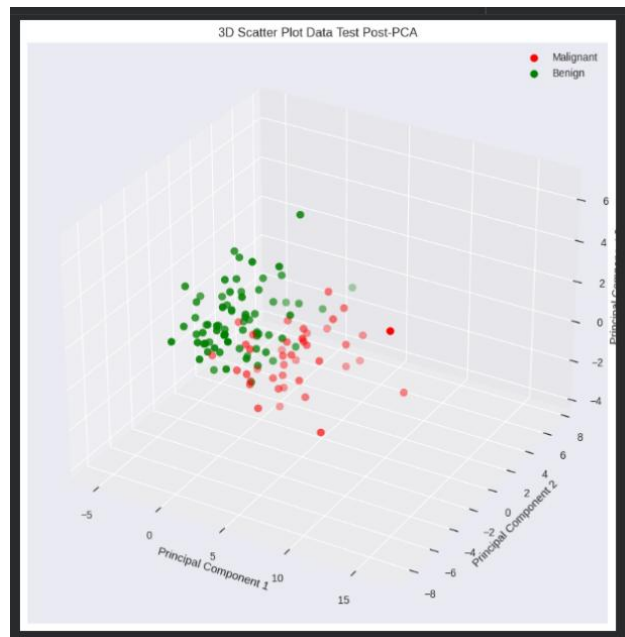
Gambar 19. Confusion Matrix

Di sini, saya memvisualisasikan confusion matrix untuk model SVM yang dilatih pada data PCA menggunakan heatmap seaborn. Hasilnya menunjukkan bahwa dari 114 sampel uji, model berhasil memprediksi benar 39 sampel Malignant dan 69 sampel Benign, dengan masing-masing 3 kesalahan prediksi (3 Malignant diprediksi Benign dan 3 Benign diprediksi Malignant). Dibandingkan baseline yang hanya memiliki 2 kesalahan total, ada sedikit peningkatan error, tetapi masih sangat minim dan dapat diterima mengingat pengurangan dimensi yang ekstrem. Confusion matrix ini memperjelas bahwa pemisahan kelas tetap baik meskipun menggunakan data tereduksi, sehingga PCA terbukti berhasil menangkap informasi utama yang diperlukan untuk klasifikasi kanker payudara.

8. Visualisasi 3D Scatter Plot



Gambar 20. Visualisasi 3D Scatter Plot



Gambar 21. 3D Scatter Plot Data Test Post PCA

Pada bagian terakhir ini, saya membuat visualisasi 3D scatter plot untuk data uji setelah direduksi dengan PCA menggunakan Matplotlib. Saya menggabungkan data test PCA (3 komponen) dengan label target asli, kemudian memetakan titik-titik dalam ruang 3D di mana sumbu X adalah PC1, Y adalah PC2, dan Z adalah PC3. Sampel Malignant (kelas 0) saya beri warna merah, sementara Benign (kelas 1) berwarna hijau. Hasil plot menunjukkan bahwa kedua kelas membentuk dua cluster yang cukup terpisah dengan baik, meskipun ada sedikit tumpang tindih di area tengah. Pemisahan yang jelas ini membuktikan bahwa 3 principal components sudah cukup untuk menangkap pola utama yang membedakan antara tumor ganas dan jinak, walaupun hanya mempertahankan sekitar 72.9% variansi total. Visualisasi ini sangat membantu untuk memahami secara intuitif mengapa model SVM masih bisa mencapai akurasi tinggi (94.74%) meskipun dimensi fitur dikurangi drastis, sekaligus menunjukkan salah satu manfaat utama PCA yaitu memudahkan eksplorasi dan visualisasi data berdimensi tinggi.

9. Perbandingan dan Kesimpulan

9. Perbandingan dan Kesimpulan				
<pre> # Perbandingan # Perbandingan akurasi before/after PCA menunjukkan bahwa reduksi dimensi dari 30 ke 3 fitur # tidak menurunkan performa model secara signifikan, sambil mengurangi waktu komputasi dan risiko overfitting. # Ini mengonfirmasi manfaat PCA sebagai preprocessing untuk dataset high-dimensional seperti Breast Cancer. # Tabel perbandingan comparison = pd.DataFrame({ 'Model': ['SVM Tanpa PCA', 'SVM dengan PCA'], 'Jumlah Fitur': [30, 3], 'Akurasi (%)': [acc_baseline, acc_pca], 'Total Varian Dijelaskan (%)': ['N/A', sum(explained_var)*100] }) print("Perbandingan Model:") print(comparison) </pre>				
Perbandingan Model:				
Model	Jumlah Fitur	Akurasi (%)	Total Varian Dijelaskan (%)	
0 SVM Tanpa PCA	30	98.245614	N/A	
1 SVM dengan PCA	3	94.736842	72.901466	

Gambar 22. Perbandingan Model

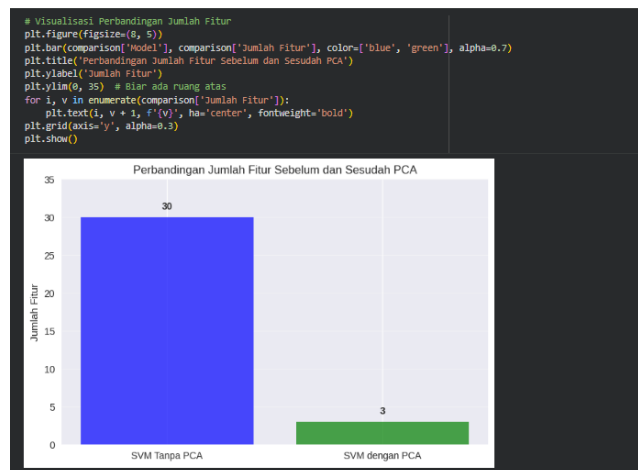
Pada bagian akhir ini, saya membuat tabel perbandingan untuk merangkum hasil keseluruhan praktikum menggunakan pandas DataFrame. Tabel ini menampilkan

dua model: SVM Tanpa PCA dengan 30 fitur yang mencapai akurasi 98.25% dan total varian N/A, serta SVM dengan PCA yang hanya menggunakan 3 fitur dengan akurasi 94.74% dan mempertahankan 72.98% total variansi data. Meskipun ada penurunan akurasi sekitar 3.5%, hasil ini masih sangat baik mengingat pengurangan dimensi yang ekstrem (dari 30 menjadi 3 fitur). Tabel ini memberikan gambaran jelas bahwa PCA berhasil menyederhanakan model secara signifikan sambil tetap menjaga performa tinggi, sehingga lebih efisien dalam hal komputasi dan mengurangi risiko overfitting pada dataset Breast Cancer yang high-dimensional.



Gambar 23. Visualisasi Perbandingan Akurasi

Di sini, saya memvisualisasikan perbandingan akurasi menggunakan bar plot, di mana bar biru mewakili SVM Tanpa PCA dengan akurasi 98.25% dan bar hijau untuk SVM dengan PCA sebesar 94.74%. Plot ini dilengkapi label nilai persen di atas setiap bar agar mudah dibaca. Visualisasi ini memperlihatkan secara langsung bahwa meskipun ada sedikit penurunan performa setelah reduksi dimensi, akurasi tetap berada di atas 94%, yang menunjukkan PCA tidak mengorbankan terlalu banyak kemampuan klasifikasi. Grafik ini memperkuat kesimpulan bahwa PCA sangat berguna sebagai teknik preprocessing untuk membuat model lebih ringkas dan cepat tanpa kehilangan efektivitas yang berarti.



Gambar 24. Visualisasi Perbandingan Jumlah Fitur

Selain akurasi, saya juga membuat bar plot untuk membandingkan jumlah fitur yang digunakan. Bar biru tinggi (30) menunjukkan SVM Tanpa PCA, sementara bar hijau pendek (3) untuk SVM dengan PCA, dengan label angka di atas masing-masing bar. Visualisasi ini secara dramatis menggambarkan pengurangan dimensi sebesar 90%, dari 30 fitur asli menjadi hanya 3 principal components. Plot ini menekankan salah satu manfaat terbesar PCA, yaitu kemampuan untuk mengompres data secara ekstrem sambil mempertahankan informasi utama, sehingga model menjadi lebih sederhana, training lebih cepat, dan lebih tahan terhadap overfitting pada data medis seperti Breast Cancer.

Link Github:

Praktikum 12:

<https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum12/notebook/praktikum12.ipynb>

Praktikum Mandiri 12:

https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum12/notebook/praktikum_mandiri12.ipynb